

Assignment 1

Name : Shravan Ranjitbhai Naika

Roll no. : 35

Subject : Node.js

Class : MSC ICT Sem-3

Q1). Develop nodejs application with following requirements:

- Develop a route `"/gethello"` with GET method. It displays **"Hello NodeJS!!"** as response.
- Make an HTML page and display.
- Call `"/gethello"` route from HTML page using AJAX call. (Any frontend AJAX call API can be used.)

➤ Index.html:

```
<!DOCTYPE html>
<html>
<head>
  <title>NodeJS AJAX Example</title>
</head>
<body>
  <h1>NodeJS AJAX Example</h1>
  <button onclick="getHello()">Get Hello</button>
  <h2 id="result"></h2>
  <script>
    function getHello() {

      fetch("/gethello")
        .then(response => response.text())
        .then(data => {
          document.getElementById("result").innerText = data;
        })
        .catch(error => {
          console.log(error);
        });
    }
  </script>
</body>
</html>
```

➤ Server.js:

```
const http = require("http");
const fs = require("fs");

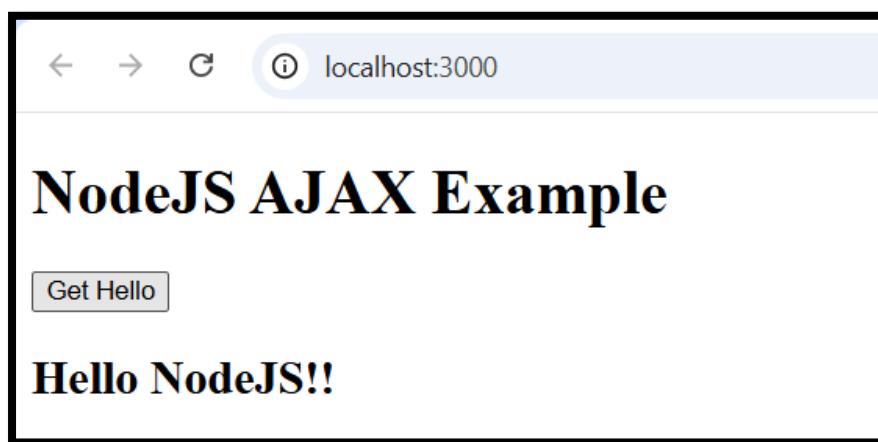
const server = http.createServer((req, res) => {

  if (req.url === "/gethello" && req.method === "GET") {

    res.writeHead(200, { "Content-Type": "text/plain" });
    res.end("Hello NodeJS!!");
```

```
}  
else if (req.url === "/" && req.method === "GET") {  
  
    fs.readFile("index.html", (err, data) => {  
  
        if (err) {  
            res.writeHead(500);  
            res.end("Error loading HTML file");  
            return;  
        }  
  
        res.writeHead(200, { "Content-Type": "text/html" });  
        res.end(data);  
    });  
  
}  
else {  
    res.writeHead(404);  
    res.end("Page Not Found");  
}  
  
});  
  
server.listen(3000, () => {  
    console.log("Server running at http://localhost:3000");  
});
```

OUTPUT:



Q2). Develop a web server which serves static resources.

➤ Index.html:

```
<!DOCTYPE html>
<html>
<head>
  <title>Static Resource Server</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Static Resource Server</h1>
  <p>This HTML page is served by NodeJS.</p>
</body>
</html>
```

➤ Server.js:

```
const http = require("http");
const fs = require("fs");
const path = require("path");
const server = http.createServer((req, res) => {
  let filePath = path.join(
    __dirname,
    req.url === "/" ? "index.html" : req.url
  );
  const extname = path.extname(filePath);

  let contentType = "text/html";

  if (extname === ".css") {
    contentType = "text/css";
  }

  fs.readFile(filePath, (err, content) => {

    if (err) {
      res.writeHead(404);
      res.end("File Not Found");
      return;
    }

    res.writeHead(200, {
      "Content-Type": contentType
    });
```

```
        res.end(content);
    });

});

server.listen(3001, () => {
    console.log("Static server running at http://localhost:3001");
});
```

OUTPUT:

Static Resource Server

This HTML page is served by NodeJS.

Q3). Develop a module for domain specific chatbot and use it in a command line application.

➤ Chatbot.js:

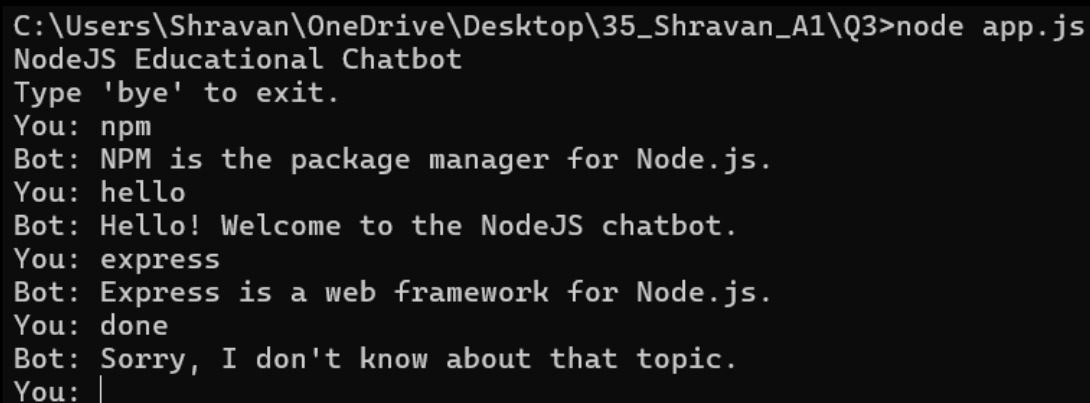
```
function getResponse(message) {  
    message = message.toLowerCase();  
    if (message.includes("node")) {  
        return "Node.js is a JavaScript runtime built on Chrome's V8 engine.";  
    }  
    if (message.includes("npm")) {  
        return "NPM is the package manager for Node.js.";  
    }  
    if (message.includes("express")) {  
        return "Express is a web framework for Node.js.";  
    }  
    if (message.includes("javascript")) {  
        return "JavaScript is a programming language used for web development.";  
    }  
    if (message.includes("hello")) {  
        return "Hello! Welcome to the NodeJS chatbot.";  
    }  
    if (message.includes("bye")) {  
        return "Goodbye!";  
    }  
    return "Sorry, I don't know about that topic.";  
}  
module.exports = {  
    getResponse  
};
```

➤ App.js:

```
const readline = require("readline");
const chatbot = require("./chatbot");
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

console.log("NodeJS Educational Chatbot");
console.log("Type 'bye' to exit.");
function askQuestion() {
  rl.question("You: ", (message) => {
    const response = chatbot.getResponse(message);
    console.log("Bot:", response);
    if (message.toLowerCase() === "bye") {
      rl.close();
    } else {
      askQuestion();
    }
  });
}
askQuestion();
```

OUTPUT:



```
C:\Users\Shravan\OneDrive\Desktop\35_Shravan_A1\Q3>node app.js
NodeJS Educational Chatbot
Type 'bye' to exit.
You: npm
Bot: NPM is the package manager for Node.js.
You: hello
Bot: Hello! Welcome to the NodeJS chatbot.
You: express
Bot: Express is a web framework for Node.js.
You: done
Bot: Sorry, I don't know about that topic.
You: |
```

Q4). Write a program to create a compressed zip file for a folder.

➤ Zip.js:

```
const fs = require("fs");
const { ZipArchive } = require("archiver");
const output = fs.createWriteStream("sample.zip");
const archive = new ZipArchive({
  zlib: { level: 9 }
});
output.on("close", () => {
  console.log("ZIP file created successfully.");
  console.log("Total bytes:", archive.pointer());
});

archive.on("error", (err) => {
  console.error("Error:", err);
});

archive.pipe(output);
archive.directory("sample/", false);
archive.finalize();
```

OUTPUT:

```
C:\Users\Shravan\OneDrive\Desktop\35_Shravan_A1\Q4>node zip.js
ZIP file created successfully.
Total bytes: 301
```

```
Directory of C:\Users\Shravan\OneDrive\Desktop\35_Shravan_A1\Q4

21-08-2026  06.35 PM    <DIR>          .
21-08-2026  06.06 PM    <DIR>          ..
21-08-2026  04.31 PM    <DIR>          sample
21-08-2026  06.35 PM                301 sample.zip
21-08-2026  04.38 PM                491 zip.js
                2 File(s)                792 bytes
                3 Dir(s)  17,586,069,504 bytes free
```


Q5). Write a program to extract a zip file.

➤ extract.js:

```
const fs = require("fs");
const unzipper = require("unzipper");
async function extractZip() {
  try {
    console.log("Starting ZIP extraction...");
    await fs.createReadStream("sample.zip")
      .pipe(unzipper.Extract({ path: "extracted" }))
      .promise();

    console.log("ZIP file extracted successfully.");
    console.log("Files are extracted into the 'extracted' folder.");

  } catch (error) {
    console.log("Error:", error.message);
  }
}

extractZip();
```

OUTPUT:

```
C:\Users\Shravan\OneDrive\Desktop\35_Shravan_A1\Q5>node extract
Starting ZIP extraction...
ZIP file extracted successfully.
Files are extracted into the 'extracted' folder.
```

Q6). Write a program to promisify fs.unlink function and call it.

➤ Delete.js:

```
const fs = require("fs");
const util = require("util");

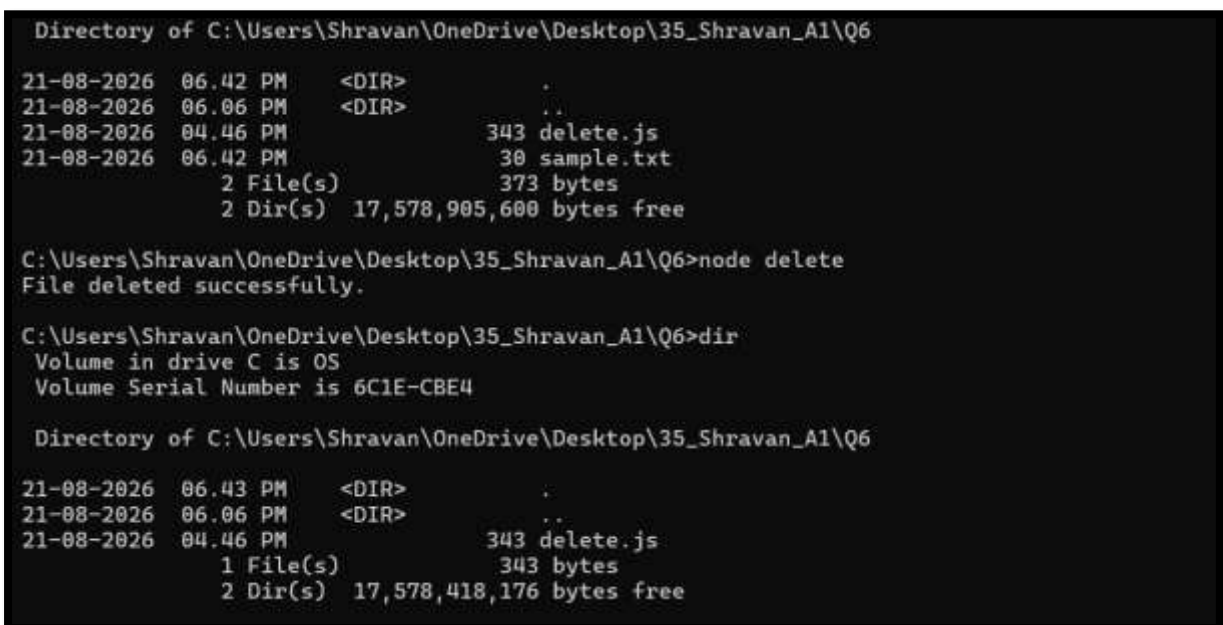
const unlinkAsync = util.promisify(fs.unlink);

async function deleteFile() {
  try {
    await unlinkAsync("sample.txt");

    console.log("File deleted successfully.");
  } catch (error) {
    console.log("Error:", error.message);
  }
}

deleteFile();
```

OUTPUT:



```
Directory of C:\Users\Shravan\OneDrive\Desktop\35_Shravan_A1\Q6
21-08-2026 06.42 PM <DIR> .
21-08-2026 06.06 PM <DIR> ..
21-08-2026 04.46 PM      343 delete.js
21-08-2026 06.42 PM      30 sample.txt
                2 File(s)      373 bytes
                2 Dir(s) 17,578,905,600 bytes free

C:\Users\Shravan\OneDrive\Desktop\35_Shravan_A1\Q6>node delete
File deleted successfully.

C:\Users\Shravan\OneDrive\Desktop\35_Shravan_A1\Q6>dir
Volume in drive C is OS
Volume Serial Number is 6C1E-CBE4

Directory of C:\Users\Shravan\OneDrive\Desktop\35_Shravan_A1\Q6
21-08-2026 06.43 PM <DIR> .
21-08-2026 06.06 PM <DIR> ..
21-08-2026 04.46 PM      343 delete.js
                1 File(s)      343 bytes
                2 Dir(s) 17,578,418,176 bytes free
```

Q7). Develop a node web server with a route /google which can fetch data of google page using fetch() using async-await model and show as a response.

➤ Server.js:

```
const http = require("http");

const server = http.createServer(async (req, res) => {

  console.log("URL RECEIVED:", req.url);

  if (req.url.startsWith("/google")) {

    try {
      console.log("Fetching Google...");

      const response = await fetch("https://www.google.com");

      const data = await response.text();

      console.log("Google data received.");
      res.writeHead(200, {
        "Content-Type": "text/html"
      });
      res.end(data);
    } catch (error) {

      console.log("FETCH ERROR:", error.message);

      res.writeHead(500, {

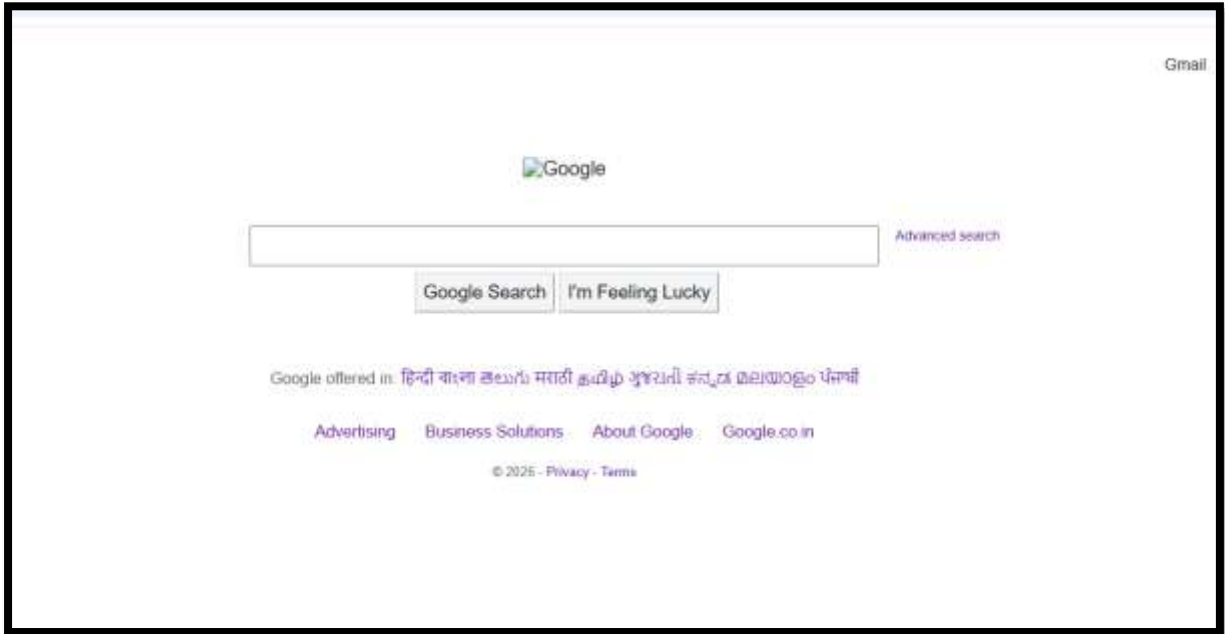
        "Content-Type": "text/plain"
      });
      res.end("Error: " + error.message);
    }

  } else {
    res.writeHead(404, {
      "Content-Type": "text/plain"
    });
    res.end("Page Not Found");
  }
});

server.listen(3002, "localhost", () => {
```

```
console.log("Open: http://localhost:3002/google");  
});
```

OUTPUT:



Q8). Set a server script, a test script and 3 user defined scripts in package.json file in your nodejs application.

➤ Server.js:

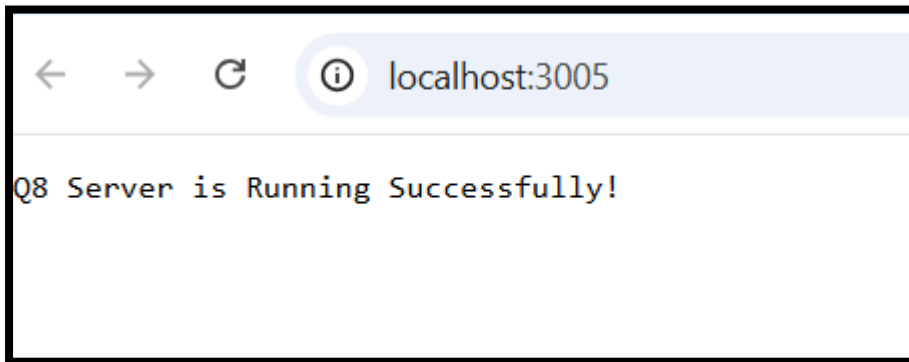
```
const http = require('http');

const server = http.createServer((req, res) => {
  res.writeHead(200, {
    'Content-Type': 'text/plain'
  });

  res.end('Q8 Server is Running Successfully!');
});

server.listen(3005, () => {
  console.log('Server running at http://localhost:3005');
});
```

OUTPUT:



Q9). A program which calls useful functions in fs module.

➤ fs_functions.js

```
const fs = require('fs');

console.log("===== FS MODULE DEMONSTRATION =====");

// 1. Write file
fs.writeFileSync(
  'sample.txt',
  'Hello NodeJS!\nThis file is created using fs module.'
);

console.log("1. File created successfully.");

// 2. Read file
const data = fs.readFileSync('sample.txt', 'utf8');

console.log("\n2. File Content:");
console.log(data);

// 3. Append data
fs.appendFileSync(
  'sample.txt',
  '\nThis line is appended using fs module.'
);

console.log("\n3. Data appended successfully.");

// 4. File information
const stats = fs.statSync('sample.txt');

console.log("\n4. File Information:");
console.log("File Size:", stats.size, "bytes");
console.log("Is File:", stats.isFile());

// 5. Rename file
fs.renameSync('sample.txt', 'new_sample.txt');

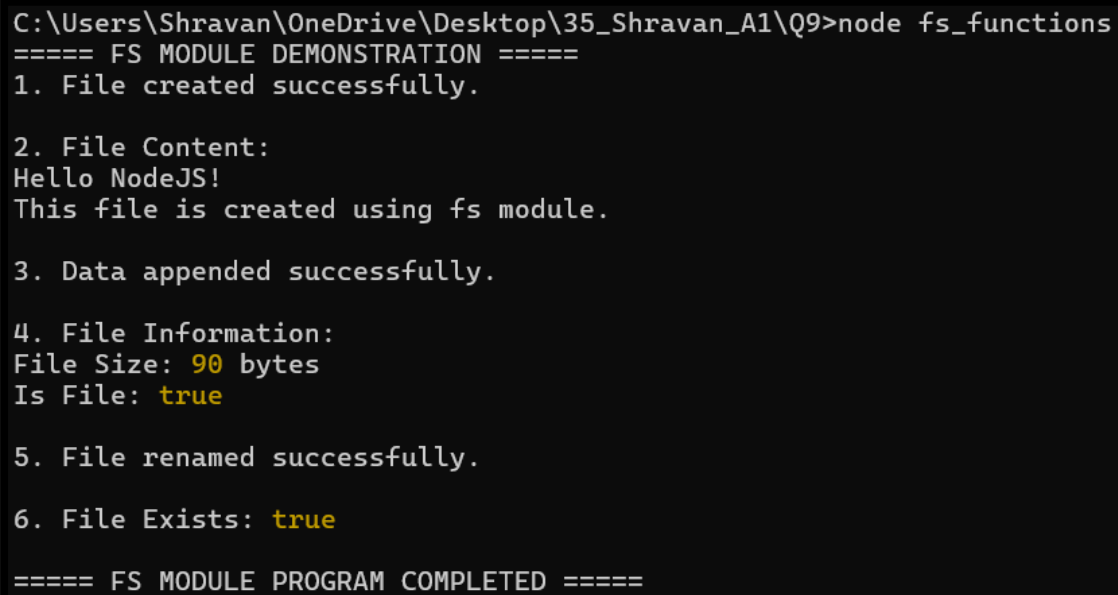
console.log("\n5. File renamed successfully.");

// 6. Check file exists
console.log(
  "\n6. File Exists:",
```

```
        fs.existsSync('new_sample.txt')
    );

    console.log("\n===== FS MODULE PROGRAM COMPLETED =====");
```

OUTPUT:



```
C:\Users\Shravan\OneDrive\Desktop\35_Shravan_A1\Q9>node fs_functions
===== FS MODULE DEMONSTRATION =====
1. File created successfully.

2. File Content:
Hello NodeJS!
This file is created using fs module.

3. Data appended successfully.

4. File Information:
File Size: 90 bytes
Is File: true

5. File renamed successfully.

6. File Exists: true

===== FS MODULE PROGRAM COMPLETED =====
```

Q10). A program which uses global objects in nodejs. Also read command line arguments and print it.

➤ Global_objects.js:

```
console.log("===== NODEJS GLOBAL OBJECTS =====");

console.log("\n1. __dirname:");
console.log(__dirname);

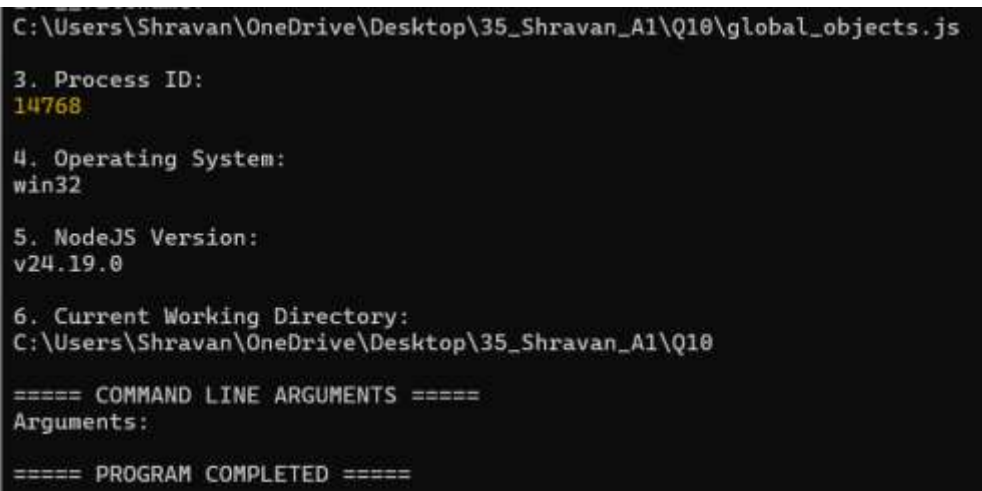
console.log("\n2. __filename:");
console.log(__filename);

console.log("\n3. Process ID:");
console.log(process.pid);
console.log("\n4. Operating System:");
console.log(process.platform);
console.log("\n5. NodeJS Version:");
console.log(process.version);
console.log("\n6. Current Working Directory:");
console.log(process.cwd());
console.log("\n===== COMMAND LINE ARGUMENTS =====");
console.log("Arguments:");

for (let i = 2; i < process.argv.length; i++) {
    console.log(`Argument ${i - 1}: ${process.argv[i]} `);
}

console.log("\n===== PROGRAM COMPLETED =====");
```

OUTPUT:



```
C:\Users\Shravan\OneDrive\Desktop\35_Shravan_A1\Q10\global_objects.js
3. Process ID:
14768

4. Operating System:
win32

5. NodeJS Version:
v24.19.0

6. Current Working Directory:
C:\Users\Shravan\OneDrive\Desktop\35_Shravan_A1\Q10

===== COMMAND LINE ARGUMENTS =====
Arguments:

===== PROGRAM COMPLETED =====
```


Q11). Develop a useful package and publish it on npmjs.com. Write a program to use it.

➤ Calculator-package/Index.js:

```
function add(a, b) {  
    return a + b;  
}  
  
function subtract(a, b) {  
    return a - b;  
}  
  
function multiply(a, b) {  
    return a * b;  
}  
  
function divide(a, b) {  
    if (b === 0) {  
        return "Cannot divide by zero";  
    }  
    return a / b;  
}  
  
module.exports = {  
    add,  
    subtract,  
    multiply,  
    divide  
};
```

➤ Calculator-package/test.js:

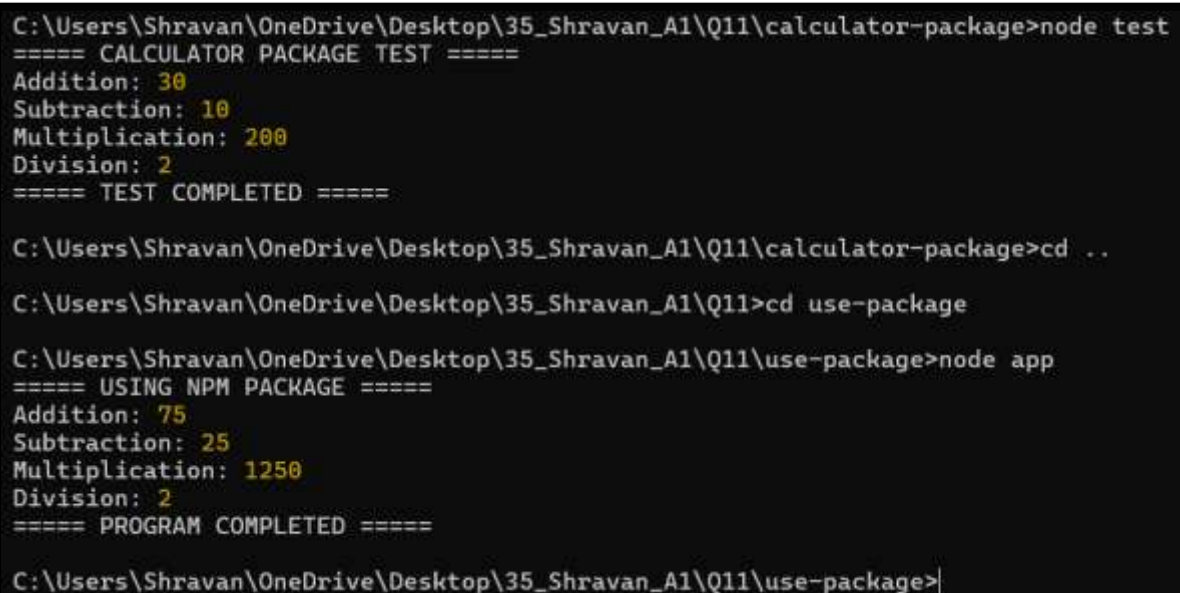
```
const calculator = require('./index');  
console.log("===== CALCULATOR PACKAGE TEST =====");
```

```
console.log("Addition:", calculator.add(20, 10));  
console.log("Subtraction:", calculator.subtract(20, 10));  
console.log("Multiplication:", calculator.multiply(20, 10));  
console.log("Division:", calculator.divide(20, 10));  
console.log("===== TEST COMPLETED =====");
```

➤ Use-package/app.js:

```
const calculator = require('shravan-simple-calculator-2026');  
console.log("===== USING NPM PACKAGE =====");  
console.log("Addition:", calculator.add(50, 25));  
console.log("Subtraction:", calculator.subtract(50, 25));  
console.log("Multiplication:", calculator.multiply(50, 25));  
console.log("Division:", calculator.divide(50, 25));  
console.log("===== PROGRAM COMPLETED =====");
```

OUTPUT:



```
C:\Users\Shravan\OneDrive\Desktop\35_Shravan_A1\Q11\calculator-package>node test  
===== CALCULATOR PACKAGE TEST =====  
Addition: 30  
Subtraction: 10  
Multiplication: 200  
Division: 2  
===== TEST COMPLETED =====  
  
C:\Users\Shravan\OneDrive\Desktop\35_Shravan_A1\Q11\calculator-package>cd ..  
  
C:\Users\Shravan\OneDrive\Desktop\35_Shravan_A1\Q11>cd use-package  
  
C:\Users\Shravan\OneDrive\Desktop\35_Shravan_A1\Q11\use-package>node app  
===== USING NPM PACKAGE =====  
Addition: 75  
Subtraction: 25  
Multiplication: 1250  
Division: 2  
===== PROGRAM COMPLETED =====  
  
C:\Users\Shravan\OneDrive\Desktop\35_Shravan_A1\Q11\use-package>
```

Q12). Write a node program having at least 3 dependent tasks and call using chained promises.

➤ chained_promises.js:

```
function task1() {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log("Task 1: User created successfully.");
      resolve("User ID: 101");
    }, 1000);
  });
}

function task2(userId) {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log("Task 2: User details fetched.");
      console.log(userId);
      resolve("Shravan");
    }, 1000);
  });
}

function task3(userName) {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log("Task 3: Welcome message generated.");
      console.log("Welcome", userName);
      resolve("All tasks completed successfully.");
    }, 1000);
  });
}

console.log("===== CHAINED PROMISES =====");

task1()
  .then((userId) => {
    return task2(userId);
  })
  .then((userName) => {
    return task3(userName);
  })
  .then((result) => {
    console.log("\nFinal Result:");
```

```
        console.log(result);
    })
    .catch((error) => {
        console.log("Error:", error);
    });
```

OUTPUT:

```
C:\Users\Shravan\OneDrive\Desktop\35_Shravan_A1\Q12>node chained_promises
===== CHAINED PROMISES =====
Task 1: User created successfully.
Task 2: User details fetched.
User ID: 101
Task 3: Welcome message generated.
Welcome Shravan

Final Result:
All tasks completed successfully.
```